1. What is Docker?

Definition

Docker is a containerization platform that allows applications and their dependencies to be packaged into isolated, portable environments called containers.

Instead of installing an application directly on the operating system, Docker allows us to package:

Application
+
Libraries
+
Dependencies
+
Configuration
+
Runtime
↓
Docker Image
↓
Docker Container

Why Docker?

Suppose you build an application on your laptop:

Application
Node.js
npm packages
Python
Database
Configuration

It works on your machine but doesn't work on another server because of different versions or missing dependencies.

Docker solves this by packaging the application and its required environment.

Main benefits

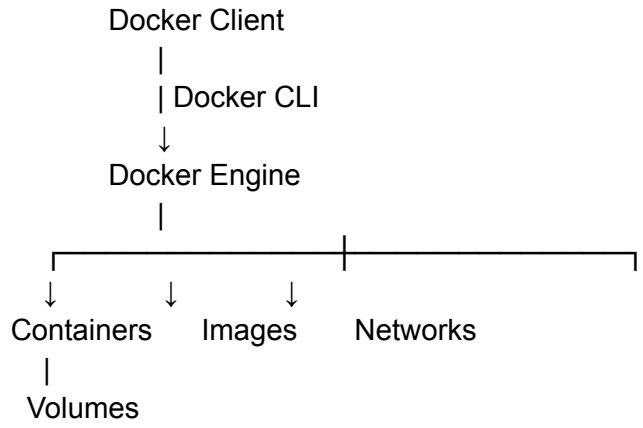
- Portability
- Isolation
- Fast deployment
- Consistent environments

- Easy scaling
- Efficient resource utilization
- Easier CI/CD
- Easy application packaging

---

2. Docker Architecture

A simplified Docker architecture:



Docker Client

The CLI through which you interact with Docker.

Example:

docker ps
docker run nginx
docker images

Docker Engine

The Docker Engine is responsible for actually managing containers, images, networks, and volumes.

Docker Image

A read-only template used to create containers.

Docker Container

A running or stopped instance of an image.

Docker Network

Allows containers to communicate with each other and with external systems.

Docker Volume

Persistent storage managed by Docker.

---

3. Docker CLI

Your notes mention the newer Docker CLI structure where commands are grouped into modules.

For example:

docker container ls

Here:

docker → main CLI
container → management module
ls → subcommand

You can see available commands using:

docker --help

For container-specific help:

docker container --help

For a specific command:

docker container run --help

This is extremely useful when learning Docker because you don't have to memorize every option.

---

4. Docker Image

Definition

A **Docker image is a read-only, versioned package containing everything required to create a container.**

For example:

nginx image
↓
Nginx software
Configuration
Required libraries
Filesystem
Metadata

An image itself isn't normally the running application.

Instead:

Docker Image
↓
docker run
↓
Docker Container

---

5. Docker Container

Definition

A **container is an isolated runtime environment created from a Docker image.**

For example:

docker container run ubuntu

Docker uses the `ubuntu` image to create a container.

Your notes correctly identify `docker container run` as the command used to create and start a container from an image.

---

6. Image vs Container

This is one of the most important Docker concepts.

Image	Container
Template	Instance

Read-only base	Runtime environment
Used to create containers	Created from image
Can be stored in registry	Can run/stop
Example: `nginx`	Example: `my-nginx`

Think about it like:

Class → Object
Image → Container

An image is like a **blueprint**.

A container is like a **running building created from that blueprint**.

---

7. Running Your First Container

docker container run ubuntu

Docker performs approximately:

1. Check whether Ubuntu image exists locally
↓
2. If not, pull image from registry
↓
3. Create container
↓
4. Start container

Your source specifically notes that if the image isn't available locally, Docker can download it from Docker Hub.

---

8. Listing Containers

Running containers

docker container ls

Shows currently running containers.

Equivalent shorthand:

docker ps

All containers

docker container ls -a

Shows:

Running
Stopped
Exited
Created

Your source covers both forms.

---

9. Container Lifecycle

A container generally goes through states like:

Created
↓
Running
↓
Stopped
↓
Started again
↓
Running
↓
Removed

Important commands:

docker container create
docker container start
docker container stop
docker container restart
docker container kill
docker container pause
docker container unpause
docker container rm

---

10. `docker run`

docker container run ubuntu

`run` essentially creates **and starts** a container.

For example:

docker container run fedora sleep 30

This starts a Fedora container and executes:

sleep 30

After 30 seconds, the command finishes and the container exits.

Your source demonstrates this lifecycle using Fedora and `sleep 30`.

---

11. Detached Mode — `-d`

docker container run -d ubuntu sleep 30

`-d` means:

> **Detached mode**

The container runs in the background and your terminal remains available.

Without `-d`:

Terminal
↓
Container running
↓
Terminal attached to process

With `-d`:

Terminal
↓
Container runs in background
↓
Terminal available

---

12. Interactive Mode — `-it`

docker container run -it ubuntu /bin/bash

Two options are commonly combined:

`-i`

Interactive.

Keeps standard input open.

`-t`

Allocates a pseudo-terminal.

Together:

-it

give you an interactive terminal inside the container.

You will see something similar to:

root@container-id:/#

Now you are working inside the container.

---

13. Exiting a Container

If you execute:

exit

or:

Ctrl + D

the shell exits.

If the shell is the container's main process, the container will normally stop as well.

Your notes also mention:

Ctrl + P
Ctrl + Q

This **detaches from the container without stopping it**, when attached interactively.

---

14. Container IDs

To get IDs of all containers:

docker container ls -aq

Breakdown:

-a → all containers
-q → quiet output, only IDs

For example:

8f31a2c
19ab321
72cd91e

You can combine this with other commands.

Example:

docker container rm $(docker container ls -aq)

This attempts to remove all containers, but **running containers must be stopped first**.

---

15. Removing Containers

docker container rm <container-id>

Removes a container.

Normally, the container must be stopped first.

To force removal:

docker container rm -f <container-id>

Remove stopped containers

docker container prune

This is safer than blindly removing every container.

---

16. Start, Stop, Restart and Kill

Start

docker container start <container-id>

Starts an existing stopped container.

Stop

docker container stop <container-id>

Gracefully stops the container.

Restart

docker container restart <container-id>

Stops and starts it again.

Kill

docker container kill <container-id>

Forcefully terminates the container process.

Difference

stop
↓
graceful termination

kill
↓
forceful termination

Your source distinguishes these two operations.

---

17. Pause and Unpause

Pause

docker container pause <container-id>

Suspends processes inside the container.

Unpause

docker container unpause <container-id>

Resumes the paused processes.

Important:

Pause is not the same as stop.

pause → processes temporarily suspended

stop → container process is terminated

---

18. Rename a Container

docker container rename <old-name> <new-name>

Example:

docker container rename old-nginx web-server

---

19. Attach vs Exec

These are frequently confused.

`docker exec`

Starts a new process inside a running container.

docker container exec -it <container-id> /bin/bash

This is generally what you want when you need a shell inside a running container.

`docker attach`

Connects your terminal to the container's existing main process.

docker container attach <container-id>

Difference

exec
↓

new process inside container

attach
↓
connect to existing main process

For troubleshooting, `exec` is usually safer.

---

20. Inspecting a Container

docker container inspect <container-id>

Displays detailed information such as:

- Container configuration
- Network settings
- IP address
- Mounts
- Environment variables
- Image
- State
- Ports
- Runtime configuration

Your source includes `inspect` as the detailed configuration command.

---

21. Container Logs

docker container logs <container-id>

Useful when your application is not working.

For example:

docker container logs nginx

You might see:

GET /
GET /index.html
404
500

Follow logs continuously

docker container logs -f <container-id>

`-f` means follow.

---

22. Processes Inside Container

docker container top <container-id>

Shows processes running inside the container.

On the host:

ps -aux

shows host processes.

---

23. Container Resource Usage

docker container stats

Displays live statistics such as:

- CPU usage
- Memory usage
- Network I/O
- Block I/O
- Process count

Very useful for monitoring containers.

---

24. Container Port Mapping

Suppose Nginx runs inside the container:

Container
  |
  | port 80
  ↓

Nginx

Port 80 exists **inside the container**.

But users outside the container need a way to access it.

We can map:

Host port 7770
↓
Container port 80

Using:

docker container run -d -p 7770:80 --name nginx-server nginx

The format is:

-p HOST_PORT:CONTAINER_PORT

So:

7770:80

means:

EC2/server port 7770
↓
Docker
↓
Container port 80
↓
Nginx

Your source specifically describes this host-to-container forwarding concept.

---

25. Example — Nginx on EC2

Suppose your EC2 instance has:

Public IP:
13.x.x.x

Run:

docker run -d -p 7770:80 --name nginx nginx

Then access:

http://13.x.x.x:7770

But there is another important layer on AWS:

Internet
↓
EC2 Public IP
↓
AWS Security Group
↓
Port 7770
↓
Docker
↓
Container port 80
↓
Nginx

Therefore, **the AWS Security Group must also allow inbound TCP 7770**.

---

26. Checking Ports

Your source uses:

netstat -nltp

This can show listening TCP ports on Linux systems where `net-tools` is installed.

Modern alternatives include:

ss -lntp

For Docker specifically:

docker container port <container-id>

This displays Docker's port mappings.

---

27. `docker wait`

docker container wait <container-id>

This waits until the container stops and then outputs its exit code.

Useful in scripts and automation.

---

28. `docker create` vs `docker run`

This is important.

`docker create`

Creates a container but does **not start it**.

docker container create ubuntu sleep 30

State:

Image
↓
Container created
↓
STOPPED/CREATED

The `sleep 30` command does not execute until the container is started.

Then:

docker container start <container-id>

`docker run`

Creates **and starts** the container.

docker container run ubuntu sleep 30

Remember

create = create only

run = create + start

Your source explicitly distinguishes these two behaviors.

---

29. Container Filesystem

Containers have their own writable filesystem layer.

For example:

```
Container
 ├── /etc
 ├── /var
 ├── /tmp
 ├── /home
 └── ...
```

If you modify files inside the container, Docker can track those changes relative to the image.

---

30. `docker diff`

docker container diff <container-id>

Shows filesystem changes made inside the container.

Common change indicators:

A = Added
C = Changed
D = Deleted

Example:

A /tmp/test.txt
C /etc/config
D /old/file

Your source uses exactly this A/C/D concept.

---

31. Monitoring Filesystem Changes

Your notes use:

watch 'docker container diff <container-id>'

This repeatedly executes the command, allowing you to observe filesystem changes.

Useful for learning and troubleshooting.

---

32. Copy Files Between Host and Container

Docker provides:

docker container cp

Example:

docker container cp test.txt mycontainer:/tmp/test.txt

This copies:

Host
↓
Container

You can also copy from container to host:

docker container cp mycontainer:/tmp/test.txt .

Your source introduces this operation under container file management.

---

33. Container Export and Import

Export

`docker container export` exports a container's filesystem as a tar archive.

Example:

docker container export <container-id> -o container.tar

Then:

ls -lh container.tar

Important concept

Container export captures the **container filesystem**, not the full image metadata/history.

---

34. Image Import

docker image import container.tar myimage:latest

This creates an image from a filesystem archive.

So:

Container
↓
docker export
↓
container.tar
↓
docker import
↓
Image

Your source covers this export/import workflow.

---

35. `docker commit`

You can create an image from an existing container:

docker container commit <container-id> myimage:latest

Example:

docker commit mycontainer my-custom-ubuntu:v1

This captures the container's current filesystem state into a new image.

But in production

Prefer a **Dockerfile** because it is:

- Repeatable
- Version-controlled
- Documented
- Reproducible
- Easier to automate

`commit` is useful for experimentation or recovering a manually modified environment.

Your source introduces `docker container commit` for this purpose.

---

36. Docker Hub

Definition

Docker Hub is a public container image registry used to store and distribute Docker images.

Think:

GitHub
↓
Source code

Docker Hub
↓
Container images

Examples:

ubuntu
nginx
mysql
redis
postgres

---

37. Pulling an Image

docker pull ubuntu

Docker downloads the image from the configured registry, normally Docker Hub when no other registry is specified.

You can specify a tag:

docker pull ubuntu:24.04

General syntax:

image:tag

Example:

ubuntu:24.04
mysql:8.4
nginx:latest

Your source covers pulling tagged images from Docker Hub.

---

38. Docker Image Tags

A tag identifies a particular version or variant of an image.

Example:

nginx:latest
nginx:1.27
nginx:alpine

You can tag an existing image:

docker image tag ubuntu myusername/myubuntu:v1

For Docker Hub:

username/repository:tag

Example:

udaymordharya/myapp:v1

---

39. Docker Login

docker login

Used to authenticate to a registry.

For Docker Hub:

docker login

Then you can push images to repositories for which you have permission.

---

40. Docker Push

docker push username/myimage:v1

Workflow:

Local Image
↓
docker tag
↓
Registry-compatible name
↓
docker push
↓
Docker Hub

---

41. Docker Image Commands

List images

docker image ls

Short form:

docker images

Your source correctly notes these are equivalent.

Inspect image

docker image inspect nginx

Image history

docker image history nginx

Shows the layers/instructions that contributed to the image.

Remove image

docker image rm nginx

Force:

docker image rm -f nginx

Remove unused images

docker image prune

---

42. Image vs Container Removal

Remember:

docker container rm

removes a **container**.

Whereas:

docker image rm

removes an **image**.

Example:

Image
↓
Container
↓
Container removed
↓
Image still exists

Removing a container doesn't automatically mean the image is removed.

---

43. Image Inspection

docker image inspect nginx

Provides information about:

- Image ID
- Architecture
- OS
- Layers
- Environment variables
- Entrypoint
- CMD
- Configuration
- Root filesystem

---

44. Image Save and Load

These commands are different from container export/import.

Save

docker image save nginx -o nginx.tar

This saves an image into an archive.

Load

docker image load -i nginx.tar

Restores the image.

Workflow:

Docker Image
↓
docker save
↓
nginx.tar
↓
Transfer to another machine
↓
docker load
↓
Docker Image

---

45. `docker save` vs `docker export`

This is a very important interview question.

`docker save`	`docker export`
Works with image	Works with container
Preserves image layers	Exports container filesystem
Preserves image metadata	Does not preserve full image metadata

Can be loaded with `docker load`	Can be imported with `docker import`
Better for transferring images	Useful for filesystem snapshot/export

Remember

Image → save → load

Container → export → import

Your source explicitly identifies this distinction as a topic.

---

46. Dockerfile

Definition

A **Dockerfile is a text file containing instructions used to build a Docker image.**

Example:

FROM ubuntu:24.04

RUN apt-get update

RUN apt-get install -y nginx

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]

Then:

docker image build -t mynginx:v1 .

---

47. Docker Build Context

In:

docker image build -t myapp:v1 .

the final:

.

means:

> Current directory is the Docker build context.

Docker can access files from this context when processing instructions such as `COPY`.

---

48. Dockerfile Layered Architecture

One of Docker's most important concepts is **layering**.

Consider:

FROM ubuntu

RUN apt update

RUN apt install -y nginx

COPY index.html /var/www/html/

Docker creates image layers based on instructions that modify the filesystem.

Conceptually:

Layer 4 → COPY index.html
Layer 3 → Install nginx
Layer 2 → apt update
Layer 1 → Ubuntu base image

The final image is composed from these layers.

Your source notes that changing instructions lower in the Dockerfile can help avoid rebuilding unaffected earlier layers.

---

49. Docker Build Cache

Docker can reuse previously built layers.

For example:

FROM ubuntu

RUN apt update

RUN apt install -y nginx

COPY index.html /var/www/html/

If you only change:

index.html

Docker can often reuse:

FROM ubuntu
RUN apt update
RUN apt install nginx

and rebuild only the later layer.

This makes builds faster.

---

50. Important Dockerfile Instructions

Your source lists:

LABEL
RUN
ENV
WORKDIR
ADD
COPY
USER
CMD
ENTRYPOINT

Let's understand each.

---

51. FROM

Although not explicitly listed in your notes, this is one of the most important Dockerfile instructions.

FROM ubuntu:24.04

Defines the base image.

Example:

FROM node:22

means the image starts from Node.js image.

---

52. RUN

Executes a command while building the image.

Example:

RUN apt-get update

Another example:

RUN apt-get install -y nginx

Important:

RUN → build time

It happens while creating the image.

---

53. CMD

Defines the default command to execute when the container starts.

Example:

CMD ["python"]

Your source specifically gives this Python example.

Another example:

CMD ["nginx", "-g", "daemon off;"]

Key idea

RUN
↓
during image build

CMD
↓
when container starts

---

54. ENTRYPOINT

`ENTRYPOINT` defines the main executable for the container.

Example:

ENTRYPOINT ["python"]

If you run:

docker run myimage script.py

Docker effectively executes:

python script.py

Your source introduces the `ENTRYPOINT` syntax but asks for the difference from CMD, so this is the key concept to remember.

---

55. CMD vs ENTRYPOINT

Very important interview topic.

CMD

Provides a default command/arguments.

CMD ["python", "app.py"]

The command can easily be overridden when running the container.

ENTRYPOINT

Defines the main executable.

ENTRYPOINT ["python"]

Then:

docker run myimage app.py

results conceptually in:

python app.py

Combined usage

ENTRYPOINT ["python"]
CMD ["app.py"]

Now:

docker run myimage

runs:

python app.py

But:

docker run myimage test.py

runs:

python test.py

Simple memory trick

ENTRYPOINT = What the container IS

CMD = Default arguments/command

---

56. ENV

Defines environment variables.

Example:

ENV APP_ENV=production

Inside the container:

echo $APP_ENV

Output:

production

Environment variables are commonly used for configuration.

However, **don't put passwords, API keys, or secrets directly into Dockerfiles**.

---

57. WORKDIR

Sets the working directory.

WORKDIR /app

Then:

COPY . .

copies files into:

/app

---

58. LABEL

Adds metadata to an image.

Example:

LABEL maintainer="uday"

Can be used for:

- Version information
- Organization
- Description
- Metadata

---

59. COPY

Copies files from the build context into the image.

Example:

COPY app.py /app/app.py

Meaning:

Host/build context
↓
app.py
↓
/app/app.py inside image

---

60. ADD

`ADD` can also copy files, but has additional behavior such as handling local tar archives and certain URL-related behavior depending on Docker's current implementation.

For most ordinary file copying:

> **Prefer `COPY` unless you specifically need `ADD` behavior.**

Your source asks for the difference between `COPY` and `ADD`.

61. COPY vs ADD

COPY	ADD
Simple file/directory copy	More features
Easier to understand	Additional behavior
Preferred for normal copying	Use when its special features are needed

Example:

COPY package.json /app/

is usually preferred over:

ADD package.json /app/

for a simple copy.

---

62. USER

Specifies the user used for subsequent Dockerfile instructions and/or container execution.

Example:

USER appuser

Instead of running the application as root.

Why?

Running applications as non-root users improves security.

Your source demonstrates creating a user and then switching to it with `USER`.

---

63. Docker Storage

Containers are designed to be relatively ephemeral.

Suppose:

MySQL container

↓

Database files

↓

Container deleted

↓

Data may be lost

To persist data, Docker provides storage mechanisms.

The three concepts from your notes are:

Docker Volume

Bind Mount

tmpfs

Your source introduces all three.

---

64. Docker Volume

Definition

A **Docker volume is persistent storage managed by Docker and stored outside the container's writable layer.**

Example:

docker volume create mydata

Then mount it:

docker run -v mydata:/var/lib/mysql mysql

Conceptually:

Docker Volume

|

↓

/var/lib/mysql

|

↓

MySQL container

If the container is deleted:

Container ❌

Volume ✅

Data ✅

---

65. Why Volumes Are Important

Imagine a MySQL container containing:

100 GB database

If you don't use persistent storage and delete the container, you risk losing the database.

With a volume:

MySQL Container

|

↓

Docker Volume

|

↓

Database data

The container can be replaced while the data remains.

This is extremely important in production.

---

66. Docker Volume Commands

Create

docker volume create mydata

List

docker volume ls

Inspect

docker volume inspect mydata

Remove

docker volume rm mydata

Remove unused volumes

docker volume prune

Your source lists this complete volume command set.

---

67. Mounting a Volume

Example from your notes:

```
docker run -d \
  --name pkmysql \
  -v abc:/var/lib/mysql \
  mysql
```

Meaning:

abc

↓

Docker volume

/var/lib/mysql

↓

Mount point inside container

So:

abc:/var/lib/mysql

means:

HOST/Docker volume : CONTAINER PATH

---

68. Important Correction — MySQL Environment Variable

Your source uses:

-e ALLOW_EMPTY_PASSWORD=true mysql

That variable is associated with some container images, but **the official MySQL image uses different environment variables**.

For the official MySQL image, common configuration includes:

-e MYSQL_ROOT_PASSWORD=secret

For example:

```
docker run -d \
  --name mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  -v mysql-data:/var/lib/mysql \
  mysql:8
```

So don't memorize `ALLOW_EMPTY_PASSWORD=true` as the general MySQL Docker syntax.

---

69. Bind Mount

Definition

A **bind mount maps a specific directory or file from the Docker host directly into the container.**

Example:

```
docker run -it \
  -v /home/root/bind:/tmp/test \
```

```
  ubuntu bash
```

Conceptually:

Host:

/home/root/bind

↓

Docker Container

↓

/tmp/test

Your source demonstrates this exact host-directory-to-container-directory concept.

---

70. Using Current Directory

You can use:

$(pwd)

to represent the current directory.

Example:

```
docker run -it \
  -v $(pwd):/tmp/test \
  ubuntu bash
```

This means:

Current host directory

↓

/tmp/test inside container

---

71. `--mount` Syntax

Instead of:

-v $(pwd):/tmp/test

you can use:

--mount type=bind,source=$(pwd),target=/tmp/test

The `--mount` syntax is more explicit and can be easier to read in complex configurations.

---

72. Volume vs Bind Mount

Volume	Bind Mount
Managed by Docker	Managed by user/host filesystem
Docker chooses storage location	User specifies host path
Good for application data	Good for source/config files
Easy to backup/manage through Docker	Direct host filesystem access
Common for databases	Common for development

Example

Database:

MySQL

↓

Docker Volume

Development:

Host source code

↓

Bind mount

↓

Container

---

73. tmpfs Mount

A **tmpfs mount stores data in the host's memory rather than persistent disk storage.**

Conceptually:

Container

↓

tmpfs

↓

RAM

When the container stops, tmpfs data does not persist.

Useful for:

- Temporary files
- Sensitive temporary information
- High-speed temporary data

Not suitable for persistent databases.

Your source specifically describes tmpfs as RAM-based storage rather than host disk storage.

---

74. Docker Networking

Docker networking allows:

- Container-to-container communication
- Container-to-host communication
- Container-to-external-network communication

Basic command:

docker network ls

Inspect a network:

docker network inspect <network>

Your source introduces Docker networking and bridge networking as the next major topic.

---

75. Docker Network Drivers

Docker supports several network drivers.

Important ones include:

bridge

host

none

overlay

macvlan

Your notes focus primarily on:

bridge

---

76. Bridge Network

Definition

A **bridge network is a Docker network that allows containers on the same Docker host to communicate with each other.**

Docker commonly provides a default bridge network.

You can create your own:

docker network create -d bridge mynetwork

Then run a container on it:

docker run -it --network mynetwork ubuntu bash

Your source demonstrates this custom bridge network workflow.

---

77. Why Create a Custom Bridge Network?

Suppose you have:

Frontend

  |

  ↓

Backend

  |

  ↓

MySQL

You can create:

docker network create app-network

Then:

frontend ─┐

          │

backend ──┼── app-network

          │

mysql ———┘

Containers attached to the same user-defined bridge network can communicate using container/service names.

For example, the backend may connect to:

mysql:3306

instead of hard-coding a container IP.

This is one reason custom Docker networks are important.

---

78. Connect Container to Network

docker network connect <network> <container>

Example:

docker network connect app-network backend

Now the container is connected to that network.

---

79. Disconnect Container

docker network disconnect <network> <container>

Removes that network connection.

Your source includes both operations.

---

80. Network Management Commands

docker network ls

docker network create

docker network inspect

docker network connect

docker network disconnect

docker network rm

docker network prune

Your source lists these network management operations.

---

81. Docker Registry

Definition

A **Docker registry is a service that stores and distributes Docker images.**

Docker Hub is a public registry.

But organizations can create their own:

Private Docker Registry

Architecture:

Developer

|

| docker push

↓

Private Registry

|

| docker pull

↓

Server / Production

This is especially useful in enterprise environments.

---

82. Local Docker Registry

Your source uses the official registry image:

```
docker run -d \
  -p 5000:5000 \
  --name myreg \
  registry
```

Now the registry is available through port:

5000

You can query:

```
curl 127.0.0.1:5000/v2/_catalog
```

Initially you may see:

```
{"repositories":[]}
```

because no repositories have been pushed yet.

Your source demonstrates this exact registry setup.

---

83. Tagging an Image for a Registry

Suppose you have:

ubuntu

You tag it:

```
docker image tag ubuntu 127.0.0.1:5000/ubuntuok
```

Now Docker understands that the target registry is:

127.0.0.1:5000

Then:

```
docker image push 127.0.0.1:5000/ubuntuok
```

---

84. Pulling from a Private Registry

docker image pull 127.0.0.1:5000/ubuntuok

Instead of Docker Hub:

docker.io

Docker contacts:

127.0.0.1:5000

---

85. Secure vs Insecure Registry

This is an important enterprise concept.

A production Docker registry should generally use:

HTTPS/TLS

An insecure registry uses plain HTTP.

Docker normally expects registries to use secure TLS connections, with special handling/configuration required for insecure registries.

Your source demonstrates configuring an insecure registry through Docker's daemon configuration.

---

86. `daemon.json`

For an explicitly configured insecure registry, Docker daemon configuration can include:

```
{
  "insecure-registries": [
    "registry.example.local:5000"
  ]
}
```

The configuration location is commonly:

/etc/docker/daemon.json

Then restart Docker.

For example:

sudo systemctl restart docker

Your source uses:

service docker restart

which may also work on systems using the appropriate service wrapper.

---

87. Why HTTPS for Registry?

Suppose you push:

Application image

+

Credentials

over an unencrypted connection.

Attackers could potentially intercept traffic.

With TLS:

Docker Client

|

| HTTPS/TLS

↓

Registry

the connection is encrypted and the server identity can be verified.

---

88. TLS / SSL Concepts

Your source uses OpenSSL to create a certificate:

```
openssl req -newkey rsa:4096 \
  -nodes \
  -sha256 \
  -keyout certs/domain.key \
  -x509 \
  -days 365 \
  -out certs/domain.crt
```

This generates:

domain.key

domain.crt

`.key`

Private key.

`.crt`

Certificate containing the public certificate information.

---

89. Registry Hostname

Your notes use:

repo.docker.local

The registry can be accessed as:

repo.docker.local:5000

The system needs to resolve:

repo.docker.local

↓

Registry server IP

For a lab environment, your source adds the mapping to:

/etc/hosts

Example:

192.168.1.50 repo.docker.local

Your source describes this hostname-resolution step after encountering the “no such host” problem.

---

90. Docker Certificate Trust

The source configures:

/etc/docker/certs.d/

and creates:

repo.docker.local:5000

with:

ca.crt

The purpose is to allow Docker to trust the registry's certificate.

Conceptually:

Docker Client

|

| TLS certificate validation

↓

Private Registry

---

91. Secure Registry Container

The source runs the registry with:

REGISTRY_HTTP_TLS_CERTIFICATE

REGISTRY_HTTP_TLS_KEY

These tell the registry where its TLS certificate and private key are located.

Conceptually:

Registry

  |

  ├── TLS Certificate

  |

  └── TLS Private Key

---

92. Registry Authentication

TLS provides encryption and server authentication, but you may also want:

User authentication

Your source uses:

HTPASSWD

for basic registry authentication.

---

93. What is `htpasswd`?

`htpasswd` creates username/password entries for HTTP Basic Authentication.

The source uses:

```
docker container run \
  --entrypoint htpasswd \
  registry \
```

-bnB pawan pass123 > auth/htpasswd

The important concepts are:

-b → batch mode

-n → output to stdout

-B → bcrypt password hashing

The resulting file contains a username and a hashed password.

Your source specifically records the use of bcrypt for password hashing.

---

94. Why Password Hashing?

You don't want the registry authentication file to contain:

pawan:pass123

Instead it stores something conceptually like:

pawan:$2y$05$.....

The password is stored as a hash.

---

95. Registry Authentication Flow

Without authentication:

Docker Client

|

| push

↓

Registry

|

↓

Image

With authentication:

Docker Client

|

| docker login

↓

Registry

|

| authenticate

↓

Credentials accepted

|

↓

docker push

---

96. `docker login`

For the private registry:

docker login repo.docker.local:5000

Enter credentials.

For example:

Username: pawan

Password: ********

Then Docker stores authentication information locally.

After login:

docker push repo.docker.local:5000/mariadb_local

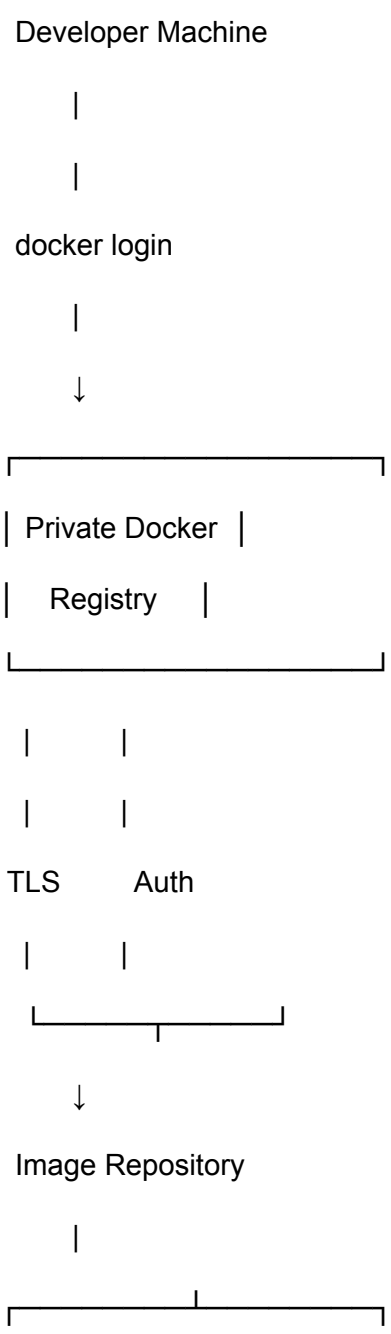
can succeed.

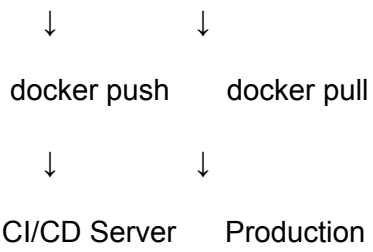
Your source demonstrates the failure before login and successful push after authentication.

---

97. Complete Private Registry Architecture

Now put everything together.

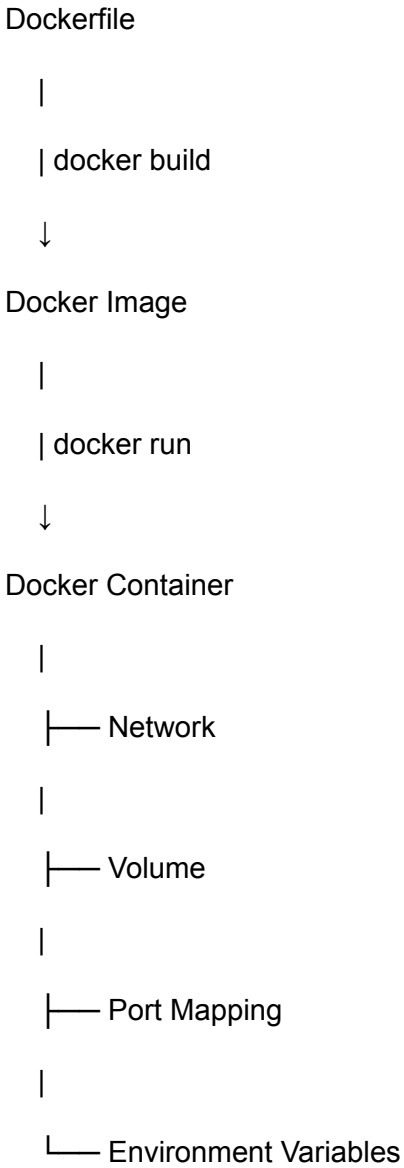




---

98. Docker Concepts You Should Connect Together

This is the most important mental model.



And:

Docker Image

|

| docker push

↓

Docker Registry

|

| docker pull

↓

Another Docker Host

---

99. Complete Docker Workflow

Suppose you're deploying an application.

Step 1 — Write application

app/

├── Dockerfile

├── package.json

└── src/

Step 2 — Create Dockerfile

FROM node:22

WORKDIR /app

COPY package*.json ./

```
RUN npm install

COPY . .

EXPOSE 3000

CMD ["npm", "start"]
```

Step 3 — Build image

```
docker build -t myapp:v1 .
```

Step 4 — Check image

```
docker image ls
```

Step 5 — Run container

```
docker run -d \
  --name myapp \
  -p 3000:3000 \
  myapp:v1
```

Step 6 — Check container

```
docker ps
```

Step 7 — Check logs

```
docker logs myapp
```

Step 8 — Test

http://server-ip:3000

Step 9 — Push to registry

```
docker tag myapp:v1 registry.example.com/myapp:v1
```

docker push registry.example.com/myapp:v1

Step 10 — Another server pulls

docker pull registry.example.com/myapp:v1

---

100. Docker Commands Cheat Sheet

Container

docker ps

docker ps -a

docker run nginx

docker run -d nginx

docker run -it ubuntu bash

docker create nginx

docker start <container>

docker stop <container>

docker restart <container>

docker kill <container>

docker pause <container>

docker unpause <container>

docker rm <container>

docker rm -f <container>

docker container prune

Container debugging

docker inspect <container>

docker logs <container>

docker logs -f <container>

docker top <container>

docker stats

docker exec -it <container> bash

docker attach <container>

docker diff <container>

docker cp <source> <container>:<destination>

docker port <container>

Images

docker images

docker image ls

docker pull nginx

docker image inspect nginx

docker image history nginx

docker image rm nginx

docker image prune

docker image tag nginx myimage:v1

docker image save nginx -o nginx.tar

docker image load -i nginx.tar

Dockerfile

docker build -t myimage:v1 .

Important instructions:

FROM

RUN

COPY

ADD

WORKDIR

ENV

LABEL

USER

EXPOSE

CMD

ENTRYPOINT

Volumes

docker volume create data

docker volume ls

docker volume inspect data

docker volume rm data

docker volume prune

Networks

docker network ls

docker network create mynetwork

docker network inspect mynetwork

docker network connect mynetwork container

docker network disconnect mynetwork container

docker network rm mynetwork

docker network prune

Registry

docker login

docker tag image registry:5000/image

docker push registry:5000/image

docker pull registry:5000/image

---

101. Most Important Interview Questions

Based on these notes, make sure you can answer these without looking at the notes.

Q1. What is Docker?

Docker is a containerization platform used to package and run applications in isolated, portable environments called containers.

Q2. What is a Docker image?

A read-only template containing the application, dependencies, filesystem, and metadata required to create containers.

Q3. What is a container?

A runtime instance created from a Docker image.

Q4. Image vs container?

Image = blueprint

Container = running instance

Q5. `docker run` vs `docker create`?

run = create + start

create = create only

Q6. What does `-d` mean?

Detached mode.

Q7. What does `-it` mean?

Interactive terminal:

-i = interactive

-t = pseudo-terminal

Q8. What does `-p 7770:80` mean?

Host port 7770

↓

Container port 80

Q9. What is a Docker volume?

Persistent Docker-managed storage.

Q10. Volume vs bind mount?

Volume → managed by Docker

Bind mount → specific host path

Q11. What is a Dockerfile?

A file containing instructions for building a Docker image.

Q12. What is a Docker layer?

A filesystem change represented as part of the image's layered structure.

Q13. Why are Docker layers useful?

They enable caching and efficient image storage/reuse.

Q14. `COPY` vs `ADD`?

`COPY` is the straightforward choice for copying files; `ADD` provides additional behavior.

Q15. `CMD` vs `ENTRYPOINT`?

ENTRYPOINT → main executable

CMD → default command/arguments

Q16. What is Docker Hub?

A public container image registry.

Q17. What is a Docker Registry?

A service that stores and distributes container images.

Q18. `docker save` vs `docker export`?

docker save → image

docker export → container filesystem

Q19. What is a bridge network?

A Docker network that enables containers on the same Docker host to communicate.

Q20. Why use a private registry?

Organizations can centrally store and distribute internal container images.

Q21. Why use TLS on a registry?

To encrypt communication and authenticate the registry endpoint.

Q22. Why use registry authentication?

To control who can push/pull images.